

CloudShield AI / DAIL

03 — DAIL Functional Specification

Version 1.0 | Final Functional Behavior Baseline | 1 September 2026

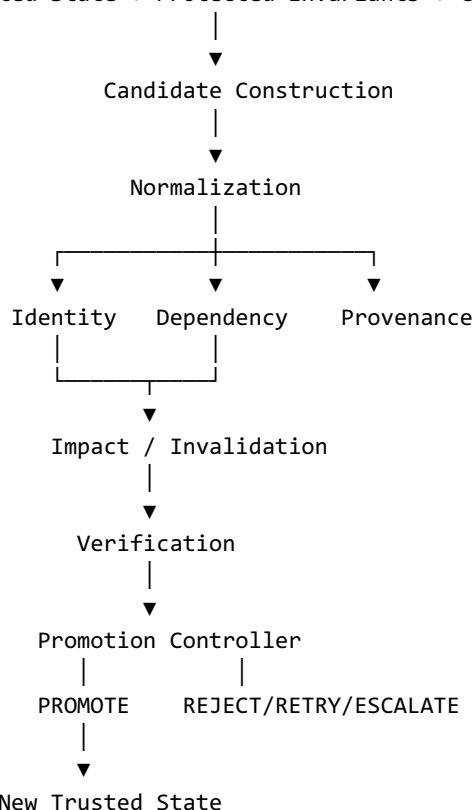
Purpose: Define the exact functional behavior of the DAIL research core at component and workflow level. This document translates the Software Requirements Specification and System Architecture Specification into operational contracts: inputs, outputs, preconditions, postconditions, processing rules, state effects, errors, uncertainty semantics, invariants, and acceptance behavior. It does not introduce a new research mechanism.

Document Control

Field	Value
Document ID	CSAI-DAIL-FUNC-003
Status	FINAL FUNCTIONAL BASELINE
Parent documents	01 — Software Requirements Specification; 02 — System Architecture Specification
Core	Dependency-Aware Invariant Ledger (DAIL)
Initial execution mode	Deterministic fixed-patch path
Later extension	LLM-generated remediation proposals
Primary scope	Controlled AWS/Terraform subset

1. Functional Model

INPUT
Trusted State + Protected Invariants + Candidate Patch



DAIL's functional unit is the candidate transition, not an isolated patch edit. The system evaluates what changed, what protected obligations may have been affected, what is currently verified, and whether the resulting state may become trusted.

2. Functional Principles

- Trusted State is the authoritative baseline for the next core evaluation.
- Candidate State is isolated and untrusted until promotion.
- Every candidate is evaluated against an explicit parent Trusted State.
- Identity, dependency, impact, verification, and promotion are separate functions.
- Impact identifies what must be checked; verification establishes whether a property currently holds.
- Uncertainty is explicit and conservative.
- Verification does not mutate state.
- Promotion does not invent evidence; it consumes explicit reports.
- Rejected candidates do not become trusted parents.
- Evidence follows the candidate transition through every stage.

3. Common Functional Contracts

3.1 Result Semantics

Result	Meaning	Promotion implication
PASS	Required property/evaluation succeeded with sufficient evidence.	May satisfy the relevant condition.
FAIL	Required property/evaluation did not hold.	Cannot satisfy the relevant condition.
UNKNOWN	Available evidence cannot establish PASS or FAIL.	Cannot satisfy a required proof condition.
UNSUPPORTED	The implementation does not support the construct/property.	Cannot satisfy a required proof condition.
VERIFIER_ERROR	The verifier could not complete reliably.	Cannot be treated as PASS; policy determines retry/escalation/rejection.

3.2 Decision Semantics

Decision	Meaning
PROMOTE	Candidate becomes the next Trusted State.
REJECT	Candidate is not accepted; Trusted State remains unchanged.
RETRY	Candidate is not accepted; another attempt may be made from the same trusted baseline.
ESCALATE	Candidate is not accepted automatically and requires higher-level/manual handling.

4. Candidate State Builder

4.1 Purpose

Create an isolated, validated Candidate State from a specific Trusted State and candidate patch.

4.2 Inputs

- Trusted State identifier/version.
- Candidate patch.
- Patch source metadata.
- Evaluation/run context.
- Terraform configuration/normalization configuration.

4.3 Outputs

- CandidateState record.
- Normalized candidate representation or construction failure.
- Parent Trusted State reference.
- Patch provenance.
- Construction diagnostics.

4.4 Preconditions

- Parent Trusted State exists and is readable.
- Patch is available as a complete evaluation input.
- Required parsing/validation tooling is available.

4.5 Processing Contract

1. Load the exact parent Trusted State.
2. Validate the patch envelope and source metadata.
3. Apply the patch to an isolated working representation.
4. Run Terraform/HCL structural validation required by the controlled subset.
5. Normalize supported resources.
6. Create candidate identity and lineage metadata.
7. Persist Candidate State as untrusted.
8. Return candidate or explicit failure.

4.6 Postconditions

- Trusted State is byte/logically unchanged.
- Candidate references exactly one parent trusted version.
- Candidate has deterministic normalized representation when construction succeeds.
- Failure does not produce a promotable candidate.

4.7 Failure Cases

Failure	Behavior
Malformed patch	Return construction failure; Trusted State unchanged.
Terraform parse/validation failure	Return failure; no downstream promotion.
Unsupported construct	Record unsupported status; candidate may continue only where the defined pipeline permits it, but unsupported semantics cannot later count as proof.
Persistence failure	Abort candidate creation cleanly; do not partially alter Trusted State.

5. Trusted State Manager

5.1 Purpose

Provide the authoritative versioned baseline and commit only successfully promoted candidates.

5.2 Core Operations

Operation	Behavior
load_current()	Return current trusted state for the evaluation context.
load(version)	Return an exact historical trusted state where available.
create_evaluation_context()	Capture parent state/version for a candidate attempt.
promote(candidate, decision_context)	Atomically commit the candidate as the next trusted state only when authorized.
verify_parent_current(version)	Prevent stale-parent overwrite during promotion.

5.3 Rules

- No component except Promotion Controller may authorize promotion.
- The manager shall reject stale-parent commits.
- Failed promotion leaves the previous trusted state active.
- Historical states remain traceable.

6. Terraform Normalization

6.1 Purpose

Convert supported Terraform source into a deterministic internal representation suitable for identity, dependency, impact, and verification.

6.2 Normalized Resource Contract

```
NormalizedResource
- resource_id / address
- resource_type
- attributes
- references
- security-relevant fields
- source/provenance metadata
- canonical representation
- fingerprint
```

6.3 Canonicalization Rules

- Object/map fields are represented in deterministic key order.
- Semantically set-like collections use deterministic ordering.
- Equivalent supported security-group rule forms normalize to one semantic form.
- Optional/default representation follows one documented canonical rule.
- Resource addresses/types are normalized consistently.
- Fingerprints are calculated from canonical representation, not raw formatting.

6.4 Functional Requirement

Two semantically equivalent supported representations must not be classified as different solely because of irrelevant Terraform syntax/ordering. Conversely, a security- or functionally-relevant difference must remain observable in the normalized model.

7. Resource Identity Engine

7.1 Purpose

Determine whether a resource in the candidate corresponds to a resource in the trusted state and classify the resulting change.

7.2 Inputs/Outputs

```
compare(trusted_resources, candidate_resources)
  → IdentityReport
```

IdentityReport:

- resource matches
- identity status
- change classification
- ambiguity/uncertainty
- evidence/rationale

7.3 Identity Semantics

Status	Functional meaning
SAME	Candidate resource corresponds confidently to the trusted resource.
DIFFERENT	Candidate resource does not correspond to the trusted resource.
UNCERTAIN	Evidence is insufficient or conflicting for safe correspondence.

7.4 Change Semantics

Classification	Meaning
UNCHANGED	Corresponding resource exists and relevant canonical representation is unchanged.
MODIFIED	Corresponding resource exists and relevant attributes changed.
CREATED	Candidate resource has no trusted counterpart.
DELETED	Trusted resource has no candidate counterpart.
REPLACED	Existing logical resource is replaced by a new identity/state according to supported rules.
UNCERTAIN	Change cannot be safely classified.

7.5 Conservative Rule

When supported evidence cannot distinguish modification from replacement, or cannot establish correspondence, the result is UNCERTAIN rather than optimistic identity reuse. Historical invariant evidence must not silently transfer across such uncertainty.

8. Dependency Graph Engine

8.1 Purpose

Build a supported typed graph connecting resources and protected invariants so impact analysis can reason beyond directly edited resources.

8.2 Node Types

- Infrastructure resource nodes.
- Protected invariant nodes.
- Optional semantic nodes only when explicitly defined by the supported model.

8.3 Edge Types

- Terraform reference/dependency.
- Infrastructure relationship.
- Invariant-resource scope relationship.
- Invariant-semantic relationship where explicitly supported.

8.4 Graph Contract

```
build(normalized_state, invariants)
    → DependencyGraphReport
```

DependencyGraphReport:

- nodes
- typed edges
- provenance
- uncertainty
- unsupported relationships

8.5 Rules

- Only supported relationships are asserted as known edges.
- Missing knowledge is not proof of absence.
- Ambiguous relationships are represented as uncertain.
- Graph construction must be deterministic for the same normalized input.

9. Impact / Invalidation Engine

9.1 Purpose

Determine the protected obligations that require current verification after a candidate transition.

9.2 Inputs

- Trusted State.
- Candidate State.
- Identity Report.
- Dependency Graph Report.
- Protected invariant registry.

9.3 Outputs

- Changed resource set.
- Affected invariant set.
- Invalidation reasons.
- Uncertainty propagation.
- Verification work set.

9.4 Processing

9. Collect CREATED, DELETED, MODIFIED, REPLACED, and UNCERTAIN resource changes.
10. Map direct resource changes to invariant scopes.
11. Traverse supported dependency edges where relevant.
12. Propagate identity/dependency uncertainty conservatively.
13. Produce the set of affected obligations requiring current verification.
14. Record reasons/evidence for each affected obligation.

9.5 Critical Distinction

AFFECTED $\neq$ VIOLATED

An affected invariant is one whose prior evidence may no longer be sufficient. Only verification can determine whether it is currently satisfied or violated.

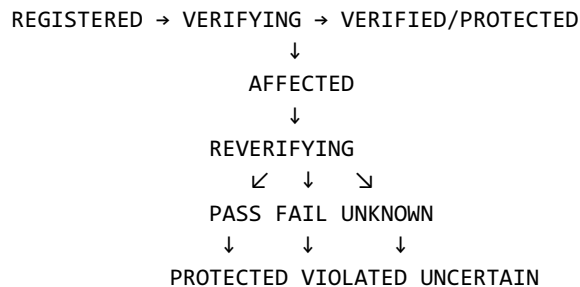
10. Invariant Store and Lifecycle

10.1 Invariant Contract

Invariant

- invariant_id
- version
- name
- category
- predicate/definition
- scope
- verifier
- status
- evidence references

10.2 Lifecycle



10.3 Functional Rules

- PROTECTED means current evidence exists for the relevant trusted state.

- A relevant candidate change invalidates stale evidence.
- Affected invariants must be reverified before being protected for the new trusted state.
- Invariant status changes are evidence-producing events.

11. Verification Engine

11.1 Purpose

Evaluate required candidate properties using the appropriate verification layers and produce explicit evidence.

11.2 Verification Layers

Layer	Function
Structural	Terraform syntax, supported model validity, plan/config consistency as required.
Security	MVP security predicates such as prohibited public exposure.
Functional	Selected functional/reachability properties such as EC2 →RDS TCP/5432.
Formal/other	Only when a later specification explicitly defines the verifier and evidence contract.

11.3 Verification Contract

```
verify(candidate, verification_work)
  → VerificationReport
```

```
VerificationReport:
- result per property
- verifier ID/version
- evidence
- execution metadata
- errors/uncertainty
```

11.4 Result Rules

- PASS requires sufficient evidence.
- FAIL requires evidence of property violation.
- UNKNOWN means the system cannot establish the property.
- UNSUPPORTED means the verifier/model does not support the property/construct.
- VERIFIER_ERROR is an execution/tooling failure and is not property evidence.

12. Canonical Security Invariant

The MVP must include at least one security predicate capable of identifying the initial security issue used by the canonical regression scenario.

```
Example invariant:
INV-SEC-001
No internet-originated inbound SSH access to protected application/database resources.
```


The exact predicate syntax and supported resource scope are defined in the dedicated Invariant Specification; this functional document only defines how such an invariant participates in the DAIL workflow.

13. Canonical Functional Invariant

The MVP must include the protected functional obligation used by the canonical regression scenario.

Example:

INV-FUNC-001

EC2 application instance can reach RDS database on TCP/5432.

This invariant is intentionally protected across repair iterations so that a security improvement cannot silently destroy an already-working functional property.

14. Promotion Controller

14.1 Purpose

Convert the complete candidate evaluation context into one controlled decision and, only for PROMOTE, authorize trusted-state advancement.

14.2 Inputs

- Candidate State.
- Parent Trusted State/version.
- Identity Report.
- Dependency Report.
- Impact Report.
- Verification Report.
- New-resource baseline checks.
- Blocking errors/uncertainty.
- Promotion policy/version.

14.3 Decision Logic

```
IF candidate structurally valid
AND new requirement PASS
AND every required affected invariant PASS
AND required new-resource baseline checks PASS
AND no blocking UNKNOWN/UNSUPPORTED/VERIFIER_ERROR
AND parent Trusted State is still current
THEN PROMOTE
ELSE apply explicit policy → REJECT / RETRY / ESCALATE
```

14.4 Postconditions

- PROMOTE: new Trusted State is committed with current evidence and lineage.
- REJECT: parent Trusted State remains active.
- RETRY: parent Trusted State remains active and a new attempt may be generated.
- ESCALATE: no trusted-state mutation.

14.5 Forbidden Behavior

- Promotion cannot ignore an affected invariant.
- Promotion cannot treat UNKNOWN as PASS.
- Promotion cannot transfer stale evidence without current verification.
- Promotion cannot use a rejected candidate as its own parent.
- Promotion cannot be triggered by the LLM.

15. End-to-End Evaluation Orchestrator

15.1 Purpose

Coordinate the component sequence while keeping business decisions inside the responsible domain components.

15.2 Pseudocode

```
evaluate_candidate(request):
    parent = trusted_state.load_current()
    candidate = candidate_builder.build(parent, request.patch)

    normalized = normalizer.normalize(candidate)
    identity = identity_engine.compare(parent, normalized)
    graph = dependency_engine.build(normalized, invariants)
    impact = impact_engine.compute(parent, normalized, identity, graph)

    verification = verifier.verify(
        candidate=normalized,
        required_work=impact.required_verification
    )

    decision = promotion_controller.decide(
        parent=parent,
        candidate=candidate,
        identity=identity,
        dependency=graph,
        impact=impact,
        verification=verification
    )

    evidence.record_all(...)

    if decision == PROMOTE:
        trusted_state.promote(candidate, decision)
    return decision
```

The orchestrator coordinates. It must not bypass domain contracts or duplicate the promotion policy.

16. New Resource Baseline Checks

A candidate that creates a new in-scope resource must not be assumed safe merely because no historical invariant covers it.

- Identify newly created in-scope resources.

- Apply applicable baseline security predicates.
- Return explicit results.
- Block promotion when a required baseline check fails or remains unproven.

17. Retry and Rejection Semantics

17.1 Retry

Retry means the current candidate was not promoted and another proposal may be evaluated. The next core attempt must use the existing trusted baseline, not the rejected candidate.

17.2 Rejection

Rejection is a terminal decision for the candidate attempt. The candidate and evidence remain available for audit, but the trusted baseline is unchanged.

17.3 Escalation

Escalation is used when automated policy cannot safely resolve the candidate, especially when uncertainty or unsupported semantics block proof.

18. Evidence Requirements Per Functional Stage

Stage	Minimum evidence
Candidate construction	Patch, source, parent state, candidate ID, validation diagnostics.
Normalization	Normalized representation, canonicalization metadata, fingerprint.
Identity	Matches, classifications, uncertainty, rationale.
Dependency	Nodes, edges, provenance, uncertainty.
Impact	Changed resources, affected invariants, invalidation reasons.
Verification	Property result, verifier/version, evidence, errors.
Promotion	Decision, policy/version, all blocking/non-blocking inputs.
Promotion commit	New state ID/version, parent state, evidence references.

19. Functional Invariants of DAIL Itself

Invariant	Required property
DAIL-F-001	Evaluating a candidate cannot mutate the active Trusted State before promotion.
DAIL-F-002	A rejected candidate cannot become a trusted parent.
DAIL-F-003	UNKNOWN cannot satisfy a proof requirement.
DAIL-F-004	UNSUPPORTED cannot satisfy a proof requirement.
DAIL-F-005	Affected protected invariants require current verification before new protection.
DAIL-F-006	Promotion must reference an exact parent Trusted State.
DAIL-F-007	Stale-parent promotion is forbidden.
DAIL-F-008	Every promoted state has traceable evidence for its decision.

20. Edge-Case Behavior

Case	Required behavior
Resource renamed but semantically same	Follow supported identity rules; if correspondence is uncertain, return UNCERTAIN.
Security rule order changes only	Canonicalization prevents false semantic change where the rules are equivalent.
Equivalent Terraform syntax	Normalize to the same representation where supported.
Resource replacement	Classify REPLACED when supported evidence establishes replacement; invalidate relevant evidence.
Unknown dependency	Propagate uncertainty into impact/verification as required.
Invariant has no verifier	UNSUPPORTED; cannot satisfy promotion proof.
Verifier crashes	VERIFIER_ERROR; cannot count as PASS.
Candidate deletes a resource covered by invariant	Affected/possibly violated; require current verification and apply promotion policy.
New resource has no historical invariant	Run applicable baseline security checks.
Parent trusted state changed during evaluation	Promotion must fail/retry rather than overwrite the newer parent.

21. Functional Test Contract

21.1 Unit Tests

- Candidate isolation.
- Canonical normalization.
- Fingerprint stability.
- Identity classifications.
- Dependency edge construction.
- Impact closure.
- Invariant lifecycle.
- Verification result semantics.
- Promotion decision matrix.

21.2 Integration Tests

- Terraform fixture → normalized model.
- Trusted state → candidate → DAIL evaluation.
- Dependency + impact + verifier integration.
- Promotion commit and rollback/failure behavior.
- Evidence reconstruction.

21.3 End-to-End Tests

- Canonical regression rejection.
- Canonical safe promotion.
- UNKNOWN blocking.
- Unsupported blocking.
- Identity uncertainty handling.
- Rejected candidate baseline isolation.
- New-resource baseline security.

22. Functional Acceptance Matrix

Capability	Acceptance condition
Candidate construction	Valid fixture produces isolated Candidate State; invalid fixture cannot mutate Trusted State.
Normalization	Equivalent supported forms produce equivalent canonical representation.
Identity	Known changes are classified correctly; ambiguity becomes UNCERTAIN.
Dependency	Known supported relationships are represented with typed edges.
Impact	Protected obligations affected by supported changes are selected for re-verification.
Verification	PASS/FAIL/UNKNOWN/UNSUPPORTED/VERIFIER_ERROR are distinct.
Promotion	Only a candidate satisfying policy becomes trusted.
Evidence	A full transition can be reconstructed from recorded evidence.
Safety	No test can demonstrate a rejected candidate becoming the trusted baseline.

23. M1–M8 Functional Delivery Mapping

Milestone	Functional output
M1	Candidate construction foundation, Terraform normalization, deterministic model/fingerprint.
M2	Trusted State + Invariant Store and lifecycle.
M3	Identity/change classification.
M4	Dependency graph.
M5	Impact/invalidation.
M6	Verification engine.
M7	Promotion Controller.
M8	Complete fixed-patch DAIL loop.
M9+	LLM, evidence, experiment harness, oracle, and research execution.

24. Functional Non-Goals

- Do not implement unrestricted Terraform semantics.
- Do not let LLM output directly alter trusted state.
- Do not infer proof from missing graph edges.
- Do not infer proof from unavailable verification.
- Do not treat impact as violation or verification.
- Do not embed experiment-specific policy inside reusable core domain logic.
- Do not add UI/API behavior that changes core DAIL semantics.

25. Implementation Readiness Criteria

- All core component contracts are explicit.
- Every cross-component input/output has a defined semantic meaning.
- State isolation and promotion authority are explicit.
- Uncertainty behavior is defined for identity, dependency, and verification.

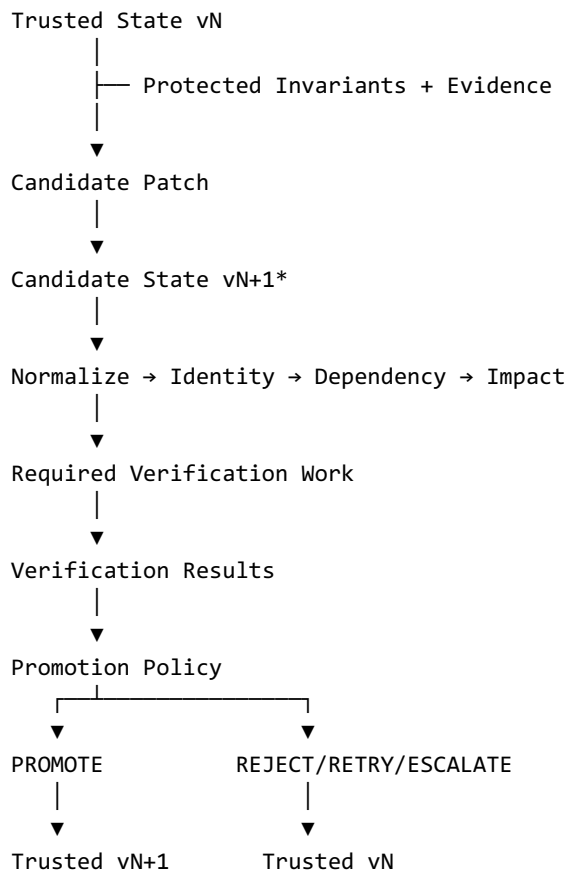
- Canonical regression and safe-promotion flows are executable as specifications.
- Functional test categories are identified.
- Remaining detailed schemas and infrastructure semantics are delegated to their dedicated specifications.

26. Final Functional Directive

DAIL shall operate as a controlled state-transition evaluator. It shall never equate a successful patch edit with a successful infrastructure repair. A candidate is acceptable only when the required new property and all currently relevant protected obligations are proven under the defined verification policy, with no blocking uncertainty or unsupported proof requirement.

The functional design deliberately separates construction, normalization, identity, dependency, impact, verification, and promotion. This separation is required for both engineering correctness and later experimental analysis.

Appendix A — Canonical Evaluation Sequence



Appendix B — Relationship to Next Specifications

Next document	Functional dependency
04 — Terraform & AWS Technical Specification	Defines exact supported resource semantics and normalization rules.
05 — Domain & Data Model Specification	Defines exact schemas and persistence mapping.
06 — State & Invariant Lifecycle Specification	Formalizes lifecycle/state transition contracts.
07 — Identity & Dependency Specification	Deepens matching and graph algorithms.
08 — Impact & Invalidation Specification	Formalizes impact closure and invalidation algorithms.

09 — Verification & Promotion Specification	Formalizes verifier and decision matrices.
---	--